

ReactJS



spring boot



Prof.º

Alexandre Gomes

>alexandre.silva251@fatec.sp.gov.br



"Java"

Hello word em JAVA

```
package com.mycompany.fundamentos;
```

```
- /**
```

```
 *
```

```
 * @author Alexandre
```

```
 */
```

```
public class Fundamentos {
```

```
-     public static void main(String[] args) {
```

```
         System.out.println("Hello World!");
```

```
     }
```

```
}
```

Palavras-Chaves do JAVA

- O Java possui 53 palavras-chaves e palavras reservadas:

abstract	class	extends	implements	null	strictfp	true
assert	const	false	import	package	super	try
boolean	continue	final	instanceof	private	switch	void
break	default	finally	int	protected	synchronized	volatile
byte	do	float	interface	public	this	while
case	double	for	long	return	throw	
catch	else	goto	native	short	throws	
char	enum	if	new	static	transient	

Algumas sequências de escape comuns.

Sequência de escape	Descrição
\n	Nova linha. Posiciona o cursor de tela no início da <i>próxima</i> linha.
\t	Tabulação horizontal. Move o cursor de tela para a próxima parada de tabulação.
\r	Retorno de carro. Posiciona o cursor da tela no início da linha <i>atual</i> — <i>não</i> avança para a próxima linha. Qualquer saída de caracteres depois do retorno de carro <i>sobrescreve</i> a saída de caracteres anteriormente gerada na linha atual.
\\	Barras invertidas. Utilizadas para imprimir um caractere de barra invertida.
\"	Aspas duplas. Utilizadas para imprimir um caractere de aspas duplas. Por exemplo, <pre>System.out.println("\"entre aspas\"");</pre> exibe "entre aspas".

Aritmética

Operação Java	Operador	Expressão algébrica	Expressão Java
Adição	+	$f + 7$	<code>f + 7</code>
Subtração	-	$p - c$	<code>p - c</code>
Multiplicação	*	bm	<code>b * m</code>
Divisão	/	x/y ou $\frac{x}{y}$ ou $x \div y$	<code>x / y</code>
Resto	%	$r \bmod s$	<code>r % s</code>

Operador(es)	Operação(ões)	Ordem de avaliação (precedência)
* / %	Multiplicação Divisão Resto	Avaliado primeiro. Se houver vários operadores desse tipo, eles são avaliados da <i>esquerda para a direita</i> .
+ -	Adição Subtração	Avaliado em seguida. Se houver vários operadores desse tipo, eles são avaliados da <i>esquerda para a direita</i> .
=	Atribuição	Avaliado por último.

Exemplo de expressões algébricas e Java

Agora vamos considerar várias expressões à luz das regras de precedência de operador. Cada exemplo lista uma expressão algébrica e seu equivalente Java. O seguinte é de uma média aritmética dos cinco termos:

Álgebra: $m = \frac{a + b + c + d + e}{5}$

Java: `m = (a + b + c + d + e) / 5;`

Álgebra: $z = pr \% q + w / x - y$

Java: `z = p * r % q + w / x - y;`

6

1

2

4

3

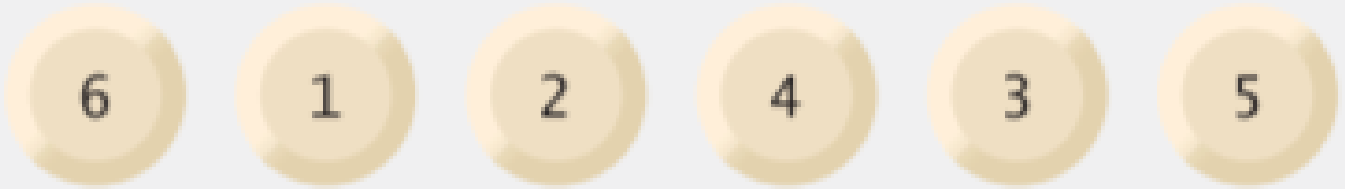
5

Avaliação de um polinômio de segundo grau

Para desenvolver um bom entendimento das regras de precedência de operadores, considere a avaliação de uma expressão de atribuição que inclui um polinômio de segundo grau $ax^2 + bx + c$:

Avaliação de um polinômio de segundo grau

Para desenvolver um bom entendimento das regras de precedência de operadores, considere a avaliação de uma expressão de atribuição que inclui um polinômio de segundo grau $ax^2 + bx + c$:

$$y = a * x * x + b * x + c;$$


The diagram illustrates the evaluation order of the expression $y = a * x * x + b * x + c;$ using numbered circles 1 through 6. The circles are positioned below the corresponding tokens in the expression:

- Circle 6 is under the variable a .
- Circle 1 is under the first multiplication operator $*$.
- Circle 2 is under the second multiplication operator $*$.
- Circle 4 is under the addition operator $+$ following the first x .
- Circle 3 is under the multiplication operator $*$ following the variable b .
- Circle 5 is under the final addition operator $+$.

The sequence of operations is: 1 (first $*$), 2 (second $*$), 4 (first $+$), 3 (second $*$), 5 (third $+$), and 6 (assignment to y).

$$y = (a * x * x) + (b * x) + c;$$

Passo 1

$$y = 2 * 5 * 5 + 3 * 5 + 7; \quad (\text{Multiplicação mais à esquerda})$$

$$2 * 5 \text{ é } 10$$

Passo 2

$$y = 10 * 5 + 3 * 5 + 7; \quad (\text{Multiplicação mais à esquerda})$$

$$10 * 5 \text{ é } 50$$

Passo 3

$$y = 50 + 3 * 5 + 7; \quad (\text{Multiplicação antes da adição})$$

$$3 * 5 \text{ é } 15$$

Passo 4

$$y = 50 + 15 + 7; \quad (\text{Adição mais à esquerda})$$

$$50 + 15 \text{ é } 65$$

Passo 5

$$y = 65 + 7; \quad (\text{Última adição})$$

$$65 + 7 \text{ é } 72$$

Passo 6

$$y = 72 \quad (\text{Última operação — coloca 72 em y})$$

SOMA

```
package com.mycompany.fundamentos;
import java.util.Scanner;

/**
 *
 * @author Alexandre
 */
public class Soma {
    public static void main(String[] args)
    {
        //cria um Scanner para obter entrada a partir da janela de comando
        Scanner input = new Scanner(System.in);
        int number1, number2, sum;

        System.out.print("Digite o primeiro numero: ");
        number1 = input.nextInt();

        System.out.print("Digite o segundo numero: ");
        number2 = input.nextInt();

        sum = number1 + number2;

        System.out.printf("A Soma é %d%n", sum);
    }
}
```

Tomada de decisão: operadores de igualdade e operadores relacionais

Operador algébrico	Operador de igualdade ou relacional Java	Exemplo de condição em Java	Significado da condição em Java
<i>Operadores de igualdade</i>			
=	==	x == y	x é igual a y
≠	!=	x != y	x é não igual a y
<i>Operadores relacionais</i>			
>	>	x > y	x é maior que y
<	<	x < y	x é menor que y
≥	>=	x >= y	x é maior que ou igual a y
≤	<=	x <= y	x é menor que ou igual a y

COMPARAÇÕES

```
package com.mycompany.fundamentos;
import java.util.Scanner;
/**
 *
 * @author Alexandre
 */
public class Comparacao {
    public static void main(String[] args){
        Scanner input = new Scanner(System.in);
        int number1, number2;

        System.out.print("Digite o primeiro numero: ");
        number1 = input.nextInt();

        System.out.print("Digite o segundo numero: ");
        number2 = input.nextInt();

        if(number1 == number2)
            System.out.printf("%d == %d\n",number1, number2);

        if(number1 != number2)
            System.out.printf("%d != %d\n",number1, number2);

        if(number1 < number2)
            System.out.printf("%d < %d\n",number1, number2);

        if(number1 > number2)
            System.out.printf("%d > %d\n",number1, number2);

        if(number1 <= number2)
            System.out.printf("%d <= %d\n",number1, number2);

        if(number1 >= number2)
            System.out.printf("%d >= %d\n",number1, number2);
    }
}
```

EXERCÍCIO

Elabore um programa que receba do usuário três números, calcule o produto desses números e imprima.

```
1 // Exercício 2.6: Product.Java
2 // Calcula o produto de três inteiros.
3 import java.util.Scanner; // programa utiliza Scanner
4
5 public class Product
6 {
7     public static void main(String[] args)
8     {
9         // cria Scanner para obter entrada a partir da janela de comando
10         Scanner input = new Scanner(System.in);
11
12         int x; // primeiro número inserido pelo usuário
13         int y; // segundo número inserido pelo usuário
14         int z; // terceiro número inserido pelo usuário
15         int result; // produto dos números
16
17         System.out.print("Enter first integer: "); // solicita entrada
18         x = input.nextInt(); // lê o primeiro inteiro
19
20         System.out.print("Enter second integer: "); // solicita entrada
21         y = input.nextInt(); // lê o segundo inteiro
22
23         System.out.print("Enter third integer: "); // solicita entrada
24         z = input.nextInt(); // lê o terceiro inteiro
25
26         result = x * y * z; // calcula o produto dos números
27
28         System.out.printf("Product is %d\n", result);
29     } // fim do método main
30 } // fim da classe Product
```

Escreva um aplicativo que exiba os números 1 a 4 na mesma linha, com cada par de adjacentes separados por um espaço. Use as seguintes técnicas:

- a) Uma instrução `System.out.println`.
- b) Quatro instruções `System.out.print`.
- c) Uma instrução `System.out.printf`.

```
package com.mycompany.fundamentos;
```

```
/**
```

```
 *
```

```
 * @author Alexandre
```

```
 */
```

```
public class Impressao {
```

```
    public static void main(String[] args) {
```

```
        //a) Uma instrução System.out.println.
```

```
        System.out.println("1 2 3 4");
```

```
        //b) Quatro instruções System.out.print
```

```
        System.out.print("1 ");
```

```
        System.out.print("2 ");
```

```
        System.out.print("3 ");
```

```
        System.out.print("4");
```

```
        //c) Uma instrução System.out.printf.
```

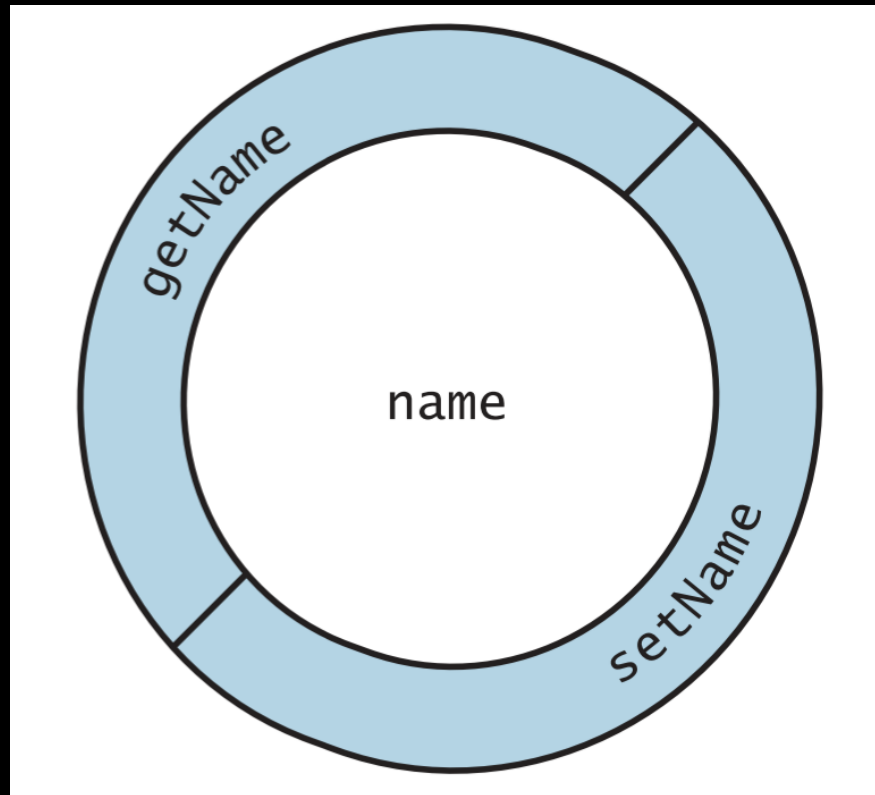
```
        System.out.printf("%n%d %d %d %d", 1, 2, 3, 4);
```

```
    }
```

```
}
```


**Variáveis de instância,
métodos set e
métodos get**

Visualização conceitual de um objeto com dados encapsulados



Inicialização de objetos com construtores

Ao declarar uma classe, você pode fornecer seu próprio construtor a fim de especificar a inicialização personalizada para objetos de sua classe. Por exemplo, você pode querer especificar um nome para um objeto “qualquer” quando ele é criado, como na linha abaixo:

```
Account account1 = new Account("Alexandre");
```

Utilizando estrutura de repetição

Tem-se um conjunto de dados contendo a altura e o sexo “M” Masculino e “F” Feminino de 5 pessoas. Fazer um programa em JAVA que calcule e escreva:

- a) A maior e a menor altura do grupo;
- b) A média de altura dos homens;
- c) A quantidade de mulheres.

O que são Arrays?

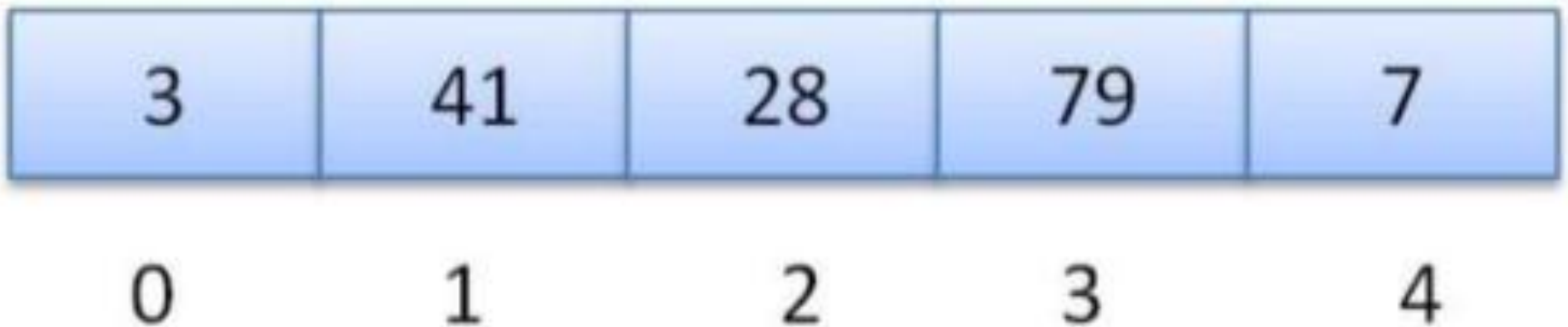


Figura 1: Um array de 5 elementos

Declaração em JAVA

```
tipo[ ] nomeDoArray = new tipo[tamanhoDoArray];
```

```
Integer[ ] numero = new Integer[10];
```

```
String[ ] nome = new String[5];
```

ARRAY

Aplicando Array

O usuário informa 5 números que são armazenados em um Array. Em seguida, o programa imprime os números informados e calcula a média deles. O programa deve usar a classe Scanner para ler os números informados pelo usuário.


```
import java.util.Scanner;
```

```
/**
```

```
 *
```

```
 * @author Alexandre
```

```
 */
```

```
public class AplicandoArray {
```

```
    public static void main(String[] args) {
```

```
        Scanner input = new Scanner(System.in);
```

```
        int[] array = new int[5];
```

```
        int soma = 0;
```

```
        double media = 0.0;
```

```
        for(int i = 0; i < array.length; i++){
```

```
            System.out.print("Digite o " + (i+1) + "º número: ");
```

```
            array[i] = input.nextInt();
```

```
            soma += array[i];
```

```
        }
```

```
        for (int num : array) {
```

```
            System.out.println(num);
```

```
        }
```

```
        media = soma / array.length;
```

```
        System.out.printf("\nMédia: %.2f\n", media);
```

```
    }
```

```
}
```

```
import java.util.ArrayList;
import java.util.Scanner;

/**
 *
 * @author Alexandre
 */
public class AplicandoArrayList {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        ArrayList<Integer> array = new ArrayList<>();
        int soma = 0;
        int numeros = 0;
        double media = 0.0;

        for(int i = 0; i < 5; i++){
            System.out.print("Digite o " + (i+1) + "º número: ");
            numeros = input.nextInt();
            array.add(numeros);
            soma += numeros;
        }

        for (int num : array) {
            System.out.println(num);
        }

        media = soma / array.size();
        System.out.printf("\nMédia: %.2f\n", media);

        input.close();
    }
}
```